

DESAFIO TÉCNICO: ARQUITETURA DE SOLUÇÕES



Arquitetura Alvo

Domínios funcionais (O Negócio)

Representam as capacidades que o usuário final ou o negócio exigem.

- **Gestão de Lançamentos (Débitos/Créditos):**
 - **O quê:** Capacidade de registrar entradas (créditos) e saídas (débitos) com data, valor e descrição.
 - **Por que:** É a atividade fim do sistema; sem o registro atômico de cada transação, não há dados para processamento.
 - API REST em .NET 8 (ASP.NET Core).
 - Banco relacional (SQL Server ou PostgreSQL).
 - Persistência via Entity Framework Core.
- **Consolidação de Saldo Diário:**
 - **O quê:** Agregação de todos os lançamentos para gerar um valor líquido final por dia.
 - **Por que:** O negócio precisa de uma visão sintetizada para tomada de decisão financeira rápida, sem ter que somar milhões de linhas em tempo real.
 - API REST em .NET 8.
 - Consome dados do serviço de lançamentos.
 - Pode usar **CQRS** para separar leitura/escrita.
 - Cache distribuído (Redis) para suportar picos de 50 req/s.

Domínios Não Funcionais (A Engenharia)

São os "Atributos de Qualidade".

Integração e Comunicação Assíncrona

Nesta etapa, o objetivo é o desacoplamento e a garantia de entrega.

Ferramenta Principal: RabbitMQ ou Azure Service Bus.

- *Por que:* O RabbitMQ é excelente para topologias complexas de roteamento (exchanges). O Azure Service Bus é a escolha natural se a empresa já estiver no ecossistema Cloud da Microsoft, pois é PaaS (gerenciado).

Abstração no .NET: MassTransit ou Rebus.

- *Por que:* Você não deve codificar "na mão" a conexão com o broker. Use o MassTransit para implementar padrões como *Retry*, *Circuit Breaker* e *Dead Letter Queues* de forma simplificada.

Escalabilidade e Orquestração

Aqui o foco é a capacidade de crescer conforme a demanda de acessos.

Runtime de Container: Docker.

- *Por que:* Padronização do ambiente de execução entre Dev e Prod.

Orquestrador: Kubernetes (AKS no Azure ou EKS na AWS).

- *Por que:* Gerencia o escalonamento horizontal automático (HPA - Horizontal Pod Autoscaler) baseado no consumo de CPU ou tamanho das filas de mensagens.

Ingress Controller / Load Balancer: NGINX Ingress ou Azure Front Door.

- *Por que:* O NGINX gerencia o tráfego dentro do cluster. O Azure Front Door atua como um WAF (Web Application Firewall) e acelerador de conteúdo global.

Monitoramento e Observabilidade

O objetivo é antecipar falhas e entender o comportamento do sistema.

Coleta de Métricas: Prometheus.

Visualização: Grafana.

Tracing (Rastreabilidade): OpenTelemetry com Jaeger ou Application Insights.

- *Por que:* Essencial para rastrear uma transação que começa na API de Lançamentos e termina no Serviço de Consolidação através do Broker.

Logs Estruturados: Serilog (dentro do .NET) enviando para um stack ELK (Elasticsearch, Logstash, Kibana) ou Azure Log Analytics.

Segurança e Identidade

Foco em proteção de dados e controle de acesso.

Identity Provider (IdP): Keycloak (Open Source) ou Microsoft Entra ID (Antigo Azure AD).

- *Por que:* Implementam nativamente OAuth2 e OpenID Connect para autenticação de usuários e comunicação *Machine-to-Machine* (M2M) entre microserviços.

Criptografia e Segredos: Azure Key Vault ou HashiCorp Vault.

- *Por que:* Chaves de API, strings de conexão e certificados nunca devem ficar no código ou no appsettings.json.

1. *Rationale & Decision Log*

ADR 001: Escolha do Broker de Mensageria

Contexto: Necessidade de integração assíncrona entre o Serviço de Lançamentos e o de Consolidação.

Decisão: Optou-se pelo **RabbitMQ** (via Amazon MQ) em detrimento do Kafka.

Justificativa: Para o volume de 50 req/s, a complexidade de gestão de partições e o overhead operacional do Kafka não trazem ROI. O RabbitMQ oferece suporte nativo a padrões complexos de roteamento (exchanges) e priorização de mensagens com menor latência de configuração.

ADR 002: Padrão de Consistência de Dados

Decisão: **Consistência Eventual** para o Saldo Consolidado.

Justificativa: Priorizou-se a **Disponibilidade (A)** e **Particionamento (P)** do teorema CAP. O negócio aceita que o saldo tenha um atraso de milissegundos para ser atualizado, desde que o sistema de registro de lançamentos nunca fique indisponível por travas (locks) no banco de dados.

2. **Matriz de Trade-offs e Atributos de Qualidade (Cross-Cutting Concerns)**

MATRIZ DE TRADE-OFFS: DECISÕES DE ARQUITETURA

Decisão Arquitetural	O que estamos 'Ganhando' (Benefícios)	O que estamos 'Perdendo' (Custos/Riscos)	Justificativa
 Arquitetura de Microserviços	Escalabilidade independente individuais, numanos operações e ramícios	Maior complexidade operacional, maior complexa conoutos	O risco de um monolito e monolito e lonioer de arquitetura
 Mensageria (RabbitMQ)	Desacoplamento total, desacoplamento e mensoserviculas	Consistência eventual, constendo um contencia alumental	Para a experiência do usuário, una experiência do usuário do MensagMQ
 CQRS (Segregação de Armadepaseo...)	Altíssima performance, performance e rudadas de performance	Duplicação lógica, lupicação lógica	Consultas de saldo, consultas de saldo em saldo
 Serviços Gerenciados...	Foco total no código, total n: Foco total no código	Custo direto, o custo de direto	O custo de manter contento de rraior do managed gerenciados
 Banco Relacional	Integridade referencial, Integridade referencial refanciaria	Menor escalabilidade, menor escalabilidade operacional	Em sistemas financeiros em sistemas financeiros atentam.

Simplicidade vs. Escalabilidade: A introdução de CQRS e Mensageria aumenta a complexidade cognitiva do código, mas é o trade-off necessário para garantir que o processamento de milhões de linhas não onere a experiência do usuário final.

Custo Operacional vs. Agilidade: O uso de serviços PaaS (RDS, ElasticCache, Amazon MQ) eleva ligeiramente o custo direto na fatura da AWS, mas reduz o custo de "Time-to-Market" e de horas/homem para manutenção de infraestrutura básica.

3. Estratégia Avançada de Resiliência (Fault Tolerance)

Resiliência na Ingestão: Implementação de **Retry com Exponential Backoff e Jitter** via Polly. Isso evita o efeito "Thundering Herd" (manada), onde todos os serviços tentam reconectar ao mesmo tempo após uma queda, derrubando o banco de dados novamente.

Ciclo de Vida de Mensagens (DLQ): Mensagens que falharem após 5 tentativas serão movidas para uma **Dead Letter Queue (DLQ)**. Foi definida uma estratégia de monitoramento (alarme CloudWatch) para que o time operacional analise e realize o *re-drive* (reprocessamento) manual ou automático após correção de bugs.

4. Segurança em Profundidade (Security by Design)

Princípio do Menor Privilégio (PoLP): Cada microsserviço possui uma **IAM Role** específica no EKS/ECS, com permissão de escrita apenas em seu respectivo schema no RDS e acesso apenas à sua fila no Broker.

Proteção de Perímetro: O **AWS WAF** será acoplado ao API Gateway para mitigar ataques de SQL Injection e Cross-Site Scripting (XSS) antes mesmo que a requisição chegue à camada de aplicação.

5. Observabilidade e Governança (SRE Mindset)

Rastreamento Distribuído (Distributed Tracing): Utilização de **Trace ID** único (via OpenTelemetry) que flui desde o API Gateway, passa pelo Broker de Mensageria e termina na persistência do Saldo. Isso permite identificar gargalos de latência em fluxos assíncronos.

Definição de SLOs (Service Level Objectives):

Disponibilidade: 99.9% para registro de débitos/créditos.

Performance: 95% das consultas de saldo (P95) devem responder em menos de 150ms.

6. Evolução e Migração de Dados

Zero-Downtime Database Migrations: Uso do padrão **Expand and Contract**. As alterações no banco de dados (Entity Framework Migrations) serão feitas de forma que a versão anterior e a nova da API possam coexistir simultaneamente, evitando erros durante o rollout dos containers.

Tabela de Stack

PROPOSTA DE STACK TECNOLÓGICA: SISTEMA DE LANÇAMENTO E CONSOLIDADO FINANCEIRO

CAMADA		TECNOLOGIA PRINCIPAL		JUSTIFICATIVA TÉCNICA (FOCO ARQUITETO)
1.	Linguagem / Framework		.NET 8 (C#)	Alta performance; LTS mais recente; Suporte nativo Linux; Ideal p/ sistemas robustos.
2.	Banco de Dados (Escrita)		PostgreSQL	Consistência ACID robusta p/ transações financeiras; Confiabilidade comprovada; Custo zero licença.
3.	Banco de Dados (Leitura/Cache)		Redis (ou DynamoDB)	Baixíssima latência (sub-milissegundo); Suporte a 50+ req/s no consolidado; Reduz carga no banco principal.
4.	Mensageria / Filas		RabbitMQ (ou Kafka)	Desacoplamento de microserviços; Garantia de entrega (não perde transação); Resiliência e assincronismo.
5.	Infraestrutura / Cloud		AWS (ou GCP)	Uso de serviços gerenciados (ex: RDS, ECS); Escalabilidade global; Ecossistema robusto.
6.	Containerização	 	Docker + Kubernetes	Portabilidade; Escalabilidade horizontal automática (HPA); Padronização do ambiente de deploy.
7.	CI/CD (Automação)		GitHub Actions	Automação completa do pipeline (build, teste, deploy); Integração nativa com repositório; Foco em qualidade.
8.	Observabilidade / Monitoramento	 	Grafana / OpenTelemetry	Rastreamento distribuído; Visibilidade de erros e performance entre serviços; Dashboards para operações.
9.	APIs e Resiliência		REST (Swagger) / Polly	Padronização e documentação; Implementação de Circuit Breaker e Retry Policies no código.



STACK FOCADA EM PERFORMANCE, BAIXO CUSTO (LINUX), E MELHORES PRÁTICAS DE SISTEMAS DISTRIBUÍDOS

Diagrama de Sequência (Autenticação e Navegação)

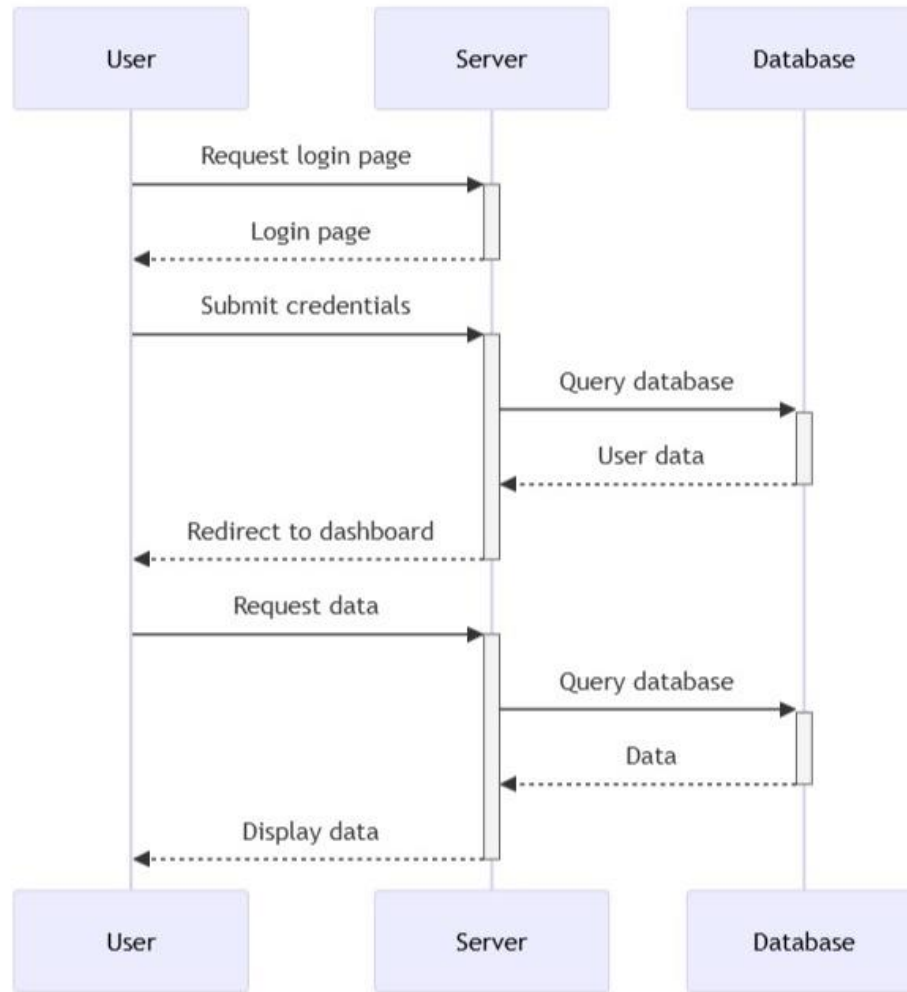


Diagrama de Sequência (Mensageria e Microserviços)

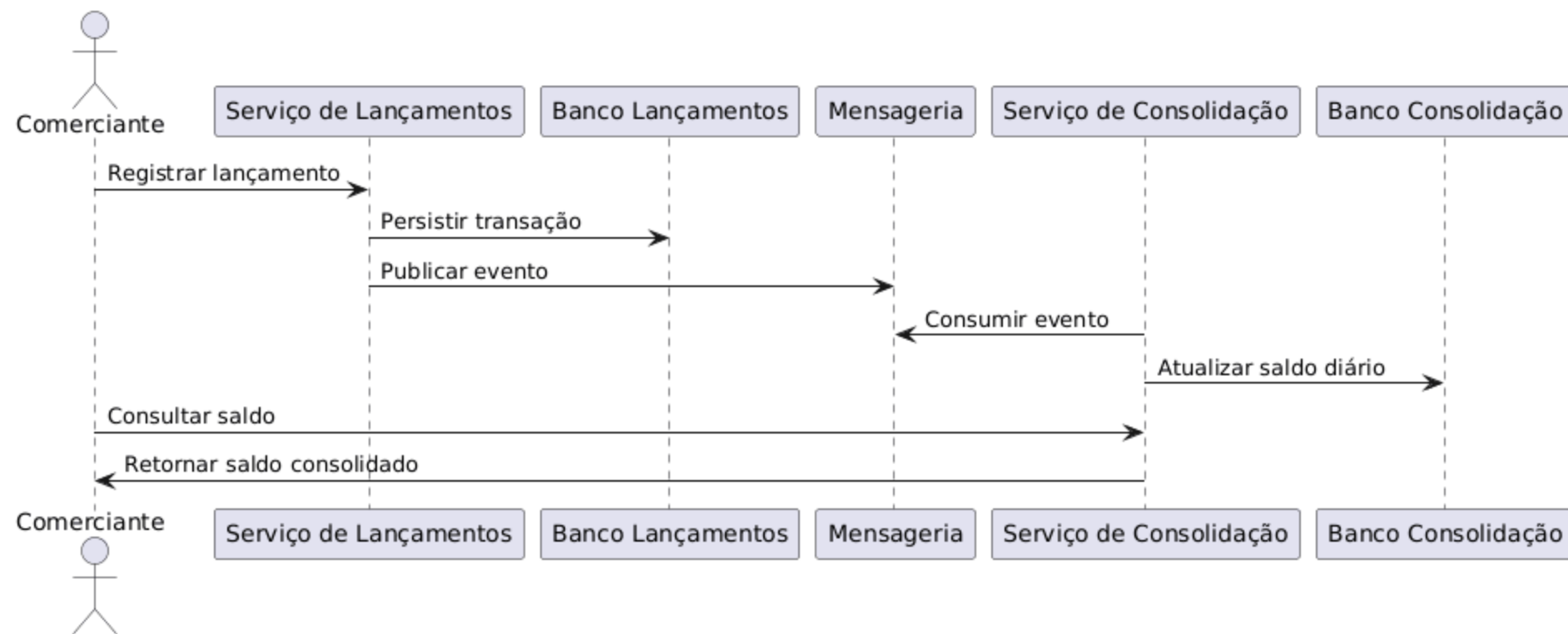
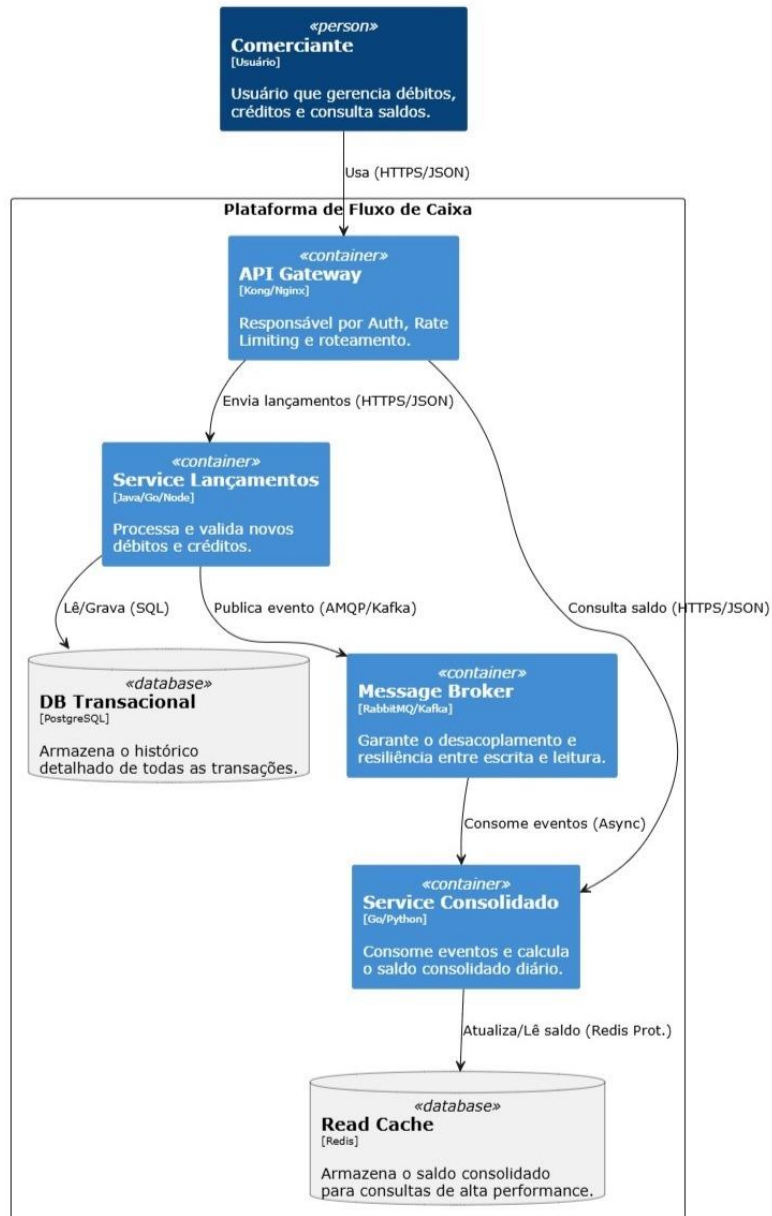


Diagrama de Container



Estimativa de Custos e Licenças (Cloud AWS)

Para uma carga de **50 requisições por segundo**, a arquitetura deve ser dimensionada para suportar picos sem desperdício. Abaixo, uma estimativa mensal baseada na região *US-East-1* (Virginia):

- Infraestrutura Gerenciada (PaaS/SaaS)**

Infraestrutura Gerenciada (PaaS/SaaS)

Recurso	Serviço Sugerido	Configuração Estimada	Custo Mensal (Est.)
Compute	AWS ECS (Fargate)	2 Microservices (2 instâncias cada p/ HA)	US\$ 60,00
Database	AWS RDS (Postgres)	db.t3.medium (Multi-AZ para resiliência)	US\$ 70,00
Mensageria	Amazon MQ (Rabbit)	t3.micro (Configuração em cluster)	US\$ 35,00
Cache/Saldo	Amazon ElastiCache	cache.t3.micro (Redis)	US\$ 15,00
Gateway	AWS API Gateway	~130 milhões de req/mês (50 req/s)	US\$ 140,00*
Total Estimado			US\$ 320,00

**Nota: O custo do API Gateway pode ser reduzido drasticamente usando um Application Load Balancer (ALB) se o orçamento for um problema.*

**Nota: O custo do API Gateway pode ser reduzido drasticamente usando um Application Load Balancer (ALB) se o orçamento for um problema.*

- **Licenciamento**

Para essa solução de arquitetura baseada em **.NET, Microserviços e Mensageria**, vou projetar os custos focando em um ambiente de produção escalável na **AWS**.

Considerando o volume de **50 req/s** (aprox. 130 milhões de requisições/mês), a estimativa é focada em serviços gerenciados para reduzir o custo operacional (PaaS).

Licenciamento: R\$ 0,00 (Open Source First)

Recomendação estratégica:

- **Runtime:** .NET 8 (Gratuito/Open Source).
- **SO:** Linux (Alpine ou Debian Slim) rodando em Containers. Isso elimina o custo de licenças Windows Server.
- **Banco de Dados:** PostgreSQL (Engine Open Source no RDS).
- **IDE:** VS Code ou Visual Studio Community/Professional (dependendo do tamanho da empresa).